ContextRuntime:
Cache-Aware Admission and Lifetime Control for Token-Efficient Coding Agents

Ankit Chauhan
InfyU Labs

Abstract

Long-horizon coding agents repeatedly re-send a growing context. Removing history can reduce context workload but may invalidate a discounted prompt cache and create a more expensive rewrite; loading all tool schemas up front can impose the same overhead on every call. We present ContextRuntime, a gateway that combines (i) pre-request admission control for unused tool schemas, (ii) forward-only retirement of superseded or cold tool results, (iii) bounded retention of prior thinking blocks, (iv) persistent mutation, and (v) a cache-aligned scheduler that fires only at a cold boundary or when expected future cache-read savings exceed the suffix rewrite cost. The gateway is off by default, supports observe-before-enforce deployment, and fails open to the original request. We evaluate the system in three evidence stages. B6 is a preregistered live A/B with 24 graded Django coding sessions: the integrated stack reduces pooled cumulative input by 41.5% (14.87M to 8.70M tokens; four-task cluster-bootstrap 95% interval 31.1–54.8%) and passes a frozen operational quality rule with 9/12 versus 10/12 successes. However, unaligned history edits increase cache writes 68.1%, leaving only 2.5% lower CLI cost. B7 fits a message-extent cache model that is exact on 11/12 native sessions and has 7.3% median creation error on mutated sessions. Replay says the aligned policy correctly holds on short, cache-hot sessions and models a 61.5% pooled dollar reduction in 36 calibrated giant interactive sessions, with zero median-session gain; this tail result is not live. B8 then preregisters a live prediction on three pairs of chained three-task sessions. Observed list-price cost falls 29.34%, within 0.16 percentage points of the frozen 29.5% prediction; CLI cost falls 29.22%, and quality is 9/9 in both arms. Crucially, the scheduler makes no profitable mid-session mutation: the B8 saving is essentially all admission. The results show that context optimization requires separate claims for workload, dollars, and task quality, and that cache-aware inaction is a first-class systems behavior.

1 Introduction

An agent that calls a language model 50 times does not pay for an early tool schema or file read once. The object remains in the request prefix and is processed on later calls, often as a discounted cache read. This creates two optimization opportunities and one trap. Admission control can prevent a large fixed object from entering at all. Lifetime control can replace an obsolete object after it has served its purpose. But a lifetime edit changes the byte prefix, potentially converting cheap cached input into an expensive cache write. With the one-hour cache tier used by the evaluated client, reads cost 0.1× base input and writes 2× [4]. Deleting context at the wrong time can lower token residency and increase dollar cost.

Prior work attacks important pieces of this problem. LLMLingua-family methods compress supplied prompts [11, 12, 23]; MemGPT and recent coding agents manage or compact memory [22, 18, 26, 33]; tool retrievers avoid exposing a large catalog [6, 8]; and serving systems reuse KV state [16, 34, 10]. Recent API-level studies directly measure prompt-cache strategy [20, 24, 15]. ContextRuntime occupies the client-side systems boundary between these areas: it does not train a compressor or control provider memory. It decides what request content to admit, what prior content can be replaced, and when a replacement is worth its cache consequence.

The design emerged from a measured elimination process, not a chosen final architecture. Search output compression was safe but too small in the whole-session denominator. A semantic code-reading surface received zero voluntary use in 11 runs. Graph retrieval did not clear a joint recall–budget gate. An offline discovery call-collapse oracle looked promising, yet its forced eager implementation raised live pooled input 55.6%. Those failures moved the system toward two mechanisms with large, prospective surfaces: shrink the fixed prefix before turn one and control the lifetime of already admitted objects. The companion measurement paper details this evidence lineage.

This paper contributes:

1. A deployable gateway that separates a forward-only safety planner from environment-specific history mutation, with observation, enforcement, persistence, recovery handles, and fail-open behavior.
2. A cache-aligned scheduling rule that compares future read savings with the cost of rewriting the divergent suffix, rather than mutating at a fixed token or turn threshold.
3. A message-extent cache simulator calibrated against live partial-prefix hits and an explicit price profile, used only for modeled claims unless a live preregistered test follows.
4. A staged evaluation: 24 live graded B6 sessions, 66-session B7 replay, and a clean B8 live validation whose cost prediction misses by only 0.16 percentage points.
5. An honest decomposition: B6 demonstrates context-workload reduction; B8 demonstrates live dollar reduction from admission and no-harm holding; the roughly

60% giant-session lifetime payoff remains modeled.

The artifacts and executable runtime are public [5]. Results come from one client/provider configuration and Django tasks, so we treat them as design-partner evidence requiring independent replication.

2 Related Work

2.1 Prompt compression and context management

Prompt compression removes low-information text or learns a compact representation. LLMLingua reports up to 20× compression; LongLLMLingua is query-aware and targets position bias; LLMLingua-2 trains a task-agnostic token classifier [11, 12, 23]. RECOMP compresses retrieved evidence and can suppress irrelevant augmentation, while ICAE and gist tokens learn latent summaries [28, 9, 21]. These methods are plausible operators inside a runtime, but a per-prompt ratio does not decide which agent objects to target or whether a query-dependent rewrite destroys a discounted prefix.

Agent-memory work supplies the semantic control plane. MemGPT uses virtual-memory tiers [22]. CAT makes context maintenance a callable tool and reports strong SWE-bench-Verified accuracy under a bounded workspace [18]; Focus gives the agent explicit checkpoint/consolidate operations and reports 22.7% fewer tokens on five instances at unchanged 3/5 success [26]; AutoCompact learns a compaction policy [33]. ContextRuntime differs in four respects: retirement can be driven by forward mechanical rules, exact payloads remain recoverable, scheduling includes provider cache economics, and failure handling preserves the original request. The approaches are complementary: learned summaries can become another mutation type governed by the same cache gate.

2.2 Tool admission and coding-agent interfaces

Large tool catalogs create a fixed-prefix tax. Re-Invoke, ToolRerank, and MCP-Zero retrieve a small candidate set instead of presenting every schema [6, 35, 8]. Provider tool search similarly loads definitions on demand and reports large reductions in initial tool context [1, 3]. Our admission filter is simpler—it disallows schemas known unused for the evaluated headless task family—but it is measured end to end and exposes a deployment interaction: Claude Code normally defers MCP tools, yet falls back to eager loading when `ANTHROPIC_BASE_URL` targets a non-first-party proxy unless tool search is explicitly configured [2].

Coding-agent research shows why task evaluation cannot be replaced by token metrics. SWE-bench uses real repository issues and objective tests [13]. SWE-agent demonstrates that the agent-computer interface changes success [29]; Agentless obtains strong results with a simpler localization–repair–validation pipeline [27]; RepoCoder, GraphCoder, and LocAgent improve repository context selection [32, 19, 7]. ContextRuntime holds the agent's tool surface and grading protocol constant between arms so that an efficiency intervention must retain issue-resolution quality.

2.3 KV caching and API economics

vLLM/PagedAttention, SGLang/RadixAttention, Prompt Cache, ChunkAttention, RAGCache, CacheBlend, Preble, and Parrot optimize memory placement, prefix sharing, prefill, or application-aware scheduling inside an inference service [16, 34, 10, 31, 14, 30, 25, 17]. A proprietary API client cannot move KV pages; it can observe usage fields and preserve or change request bytes.

The closest concurrent work studies this boundary directly. Don't Break the Cache compares cache strategies across three providers and more than 500 research-agent sessions [20]. CAPC combines query-agnostic compression with explicit cache control and a two-tier cost model [24]. Keepalive economics analyzes whether paying to refresh a prefix across idle gaps is rational [15]. ContextRuntime adds coding-agent object lineage, mutation/recovery safety, partial interior-hit calibration, and a live sequence that separates admission from lifetime payoff.

3 System Design

3.1 Goals and non-goals

The gateway follows six requirements.

1. **Preserve task semantics.** Exact edit preconditions and the most recent object for a warm key remain; a retired payload is replaced by a recovery handle.
2. **Use only forward information.** Production decisions cannot use the future labels available to an offline oracle.
3. **Separate safety from timing.** The planner decides *what* is eligible; the cache scheduler decides *when* to expose a new byte divergence.
4. **Keep the mutated stream stable.** Once a mutation fires, it is reapplied to every subsequent request so history does not snap back.
5. **Fail open.** Parsing, logging, or upstream rejection must not block the agent; the unmodified body is the recovery path.
6. **Expose evidence grades.** Live, modeled, and counterfactual results remain distinct.

We do not train a semantic compressor, change the model, promise cross-provider cache equivalence, or claim that discarded content is universally irrelevant. The runtime is a policy and accounting layer.

3.2 Admission control

Tool schemas are unusually leveraged because they are fixed, early, and often unused. The evaluated admission profile uses the client's `-disallowedTools` mechanism to remove definitions that never appeared in the headless task traces. A zero-model-call capture reduced the transmitted tool set from 82 schemas (46.1k cl100k-estimated tokens) to six (5.6k). The filter is configured outside the model and therefore does not require voluntary tool-search adoption.

Admission is conservative. The six retained tools provide repository reading, search, editing, and execution needed by the task harness. The system never silently removes a schema selected by the active profile. Other deployments should derive their own allowlist from observe-mode

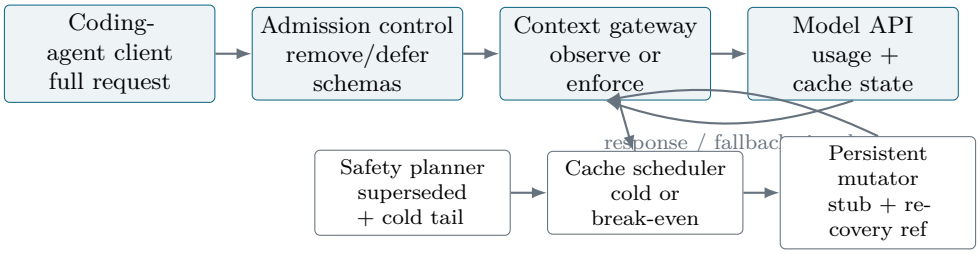

Figure 1: ContextRuntime request path. Admission changes the first request and remains prefix-stable. The gateway reconstructs objects, asks the forward planner what is eligible, asks the scheduler whether a new cache divergence pays, and applies the fired set persistently. Observation mode logs the same decision without mutation; all exception paths preserve the original request.

traces; the particular 82-to-6 profile is not a universal recommendation.

3.3 Forward retirement planner

The planner observes tool-result objects in call order. Each object has an identifier, admission turn, estimated token size, a supersession key such as `path:module.py` or a command signature, and an optional recovery reference. A later object with the same key makes earlier objects mechanically *superseded*. The most recent object for a key is retained while the key is warm. If a key has not been touched for $L = 5$ turns, its older object becomes a *cold-tail* candidate. Plans are normally batched at $K = 10$ turns to amortize rewrites.

The mutator replaces an eligible `tool_result` with a small explicit stub, not an invented summary. The stub names a content-addressed handle or instructs the agent to rerun/re-read the source. The planner is pure and environment-independent. A subscription client that cannot rewrite history uses an unsupported mutator; a gateway or custom loop that owns the message array uses the in-process mutator. This seam prevents an offline policy from being confused with a deployable mechanism.

3.4 Thinking lifetime

Models can retain prior thinking or redacted-thinking blocks in later requests. A keep-N rule removes those blocks from all but the most recent N assistant messages; the experiments use $N = 1$. Thinking removal is tracked separately because displayed transcript text can omit the payload while provider usage still reflects it. Under cache alignment, a thinking frontier advances only when a mutation fires, making the edited byte stream stable between fires.

3.5 Cache-aligned scheduler

Let G be the tokens in new pending retirements, S the estimated suffix tokens that would need to be recreated after the earliest edit point, E the expected remaining calls, r the cache-read price, and w the cache-write price. A mutation clears the break-even gate when

$$rGE \geq (w - r)S. \tag{1}$$

The default evaluated profile uses $r = 0.1$, $w = 2.0$, and $E = 8$. If each retired token requires one suffix token to be rewritten, a write requires 19 future reads to break even. Equation 1 can still fire with $E < 19$ when one early divergence retires substantially more pending content than the suffix it rewrites.

The scheduler has three modes: `off`, `cold`, and `gated`; the default is off. Cold starts are safe moments to establish the mutated stream. A TTL-gap rule can also fire when the cache is believed expired, although B8 v1 reveals that nominal TTL is soft at 65 minutes; deployments should prefer observed miss telemetry or a conservative margin. Gated mode adds Equation 1. The scheduler decides timing only and cannot promote an unsafe object.

The key state invariant is persistence. A fired identifier enters a set and is rewritten on every later request. Reapplication is byte-stable and does not create another divergence. Only newly eligible objects can cause a new cache invalidation. This repairs the B6 "flash" behavior, in which retirement applied only on batch-boundary requests and history returned on intervening calls.

3.6 Operational safety

Modes form an escalation path: off passes through, observe logs opportunities, and enforce mutates. Unexpected parsing errors return the untouched request. The live harness also retries an original body if an upstream response rejects a mutated request. Logs record the eligible count, tokens, boundary, fired reason, gap, pending mass, suffix estimate, persistent reapplications, applied retirements, stripped thinking, and fallback count. These fields turn "no harm" from an assumption into a measurable endpoint.

4 Evaluation Method

4.1 Research questions

RQ1:
Does integrated admission and lifetime control reduce cumulative live input while preserving objectively graded coding quality?

RQ2:
Can a calibrated cache model identify when lifetime mutation earns dollars and when it should hold?

RQ3:
Does a preregistered live experiment validate the model and the no-harm behavior of the aligned stack?

4.2 B6: integrated live A/B

B6 contains four modern Django SWE-bench-Verified instances, three interleaved paired repetitions, and two arms, for 24 live sessions. Every run uses the same Sonnet model,

prompt, budget cap, and fresh worktree. The native arm is stock headless Claude Code. The treatment arm applies 82-to-6 schema admission plus gateway enforcement with lag-five/batch-ten retirement and thinking keep-one. Graph, discovery, and search-replacement mechanisms are excluded because earlier gates closed them.

The preregistered primary endpoint is

$$R = \frac{\sum_t P_{t,\mathrm{T}}}{\sum_t P_{t,\mathrm{N}}}, \quad P_t = R_t + W_t + U_t, \tag{2}$$

reported per task and pooled. API calls are reconstructed by merging transcript records with the same `requestId`; sidechains are excluded. A run succeeds only when official FAIL_TO_PASS tests pass and PASS_TO_PASS tests remain green. The frozen operational quality rule requires total treatment successes to be at least native successes minus one, with no task at 0/3 treatment when native is 3/3. This is a program decision rule, not a statistically powered non-inferiority trial.

No session times out or reaches the budget cap. We add a post-hoc task-cluster bootstrap interval for effect-scale uncertainty; four tasks make it intentionally coarse, and the preregistered pooled ratio remains the decision endpoint.

4.3 B7: calibrated replay

B7 simulates cached prefix extents with TTLs and partial interior hits. On append-only call t, the largest alive matching extent is read and the remaining suffix is created. When history first differs at depth c, the request may read the largest matching prefix before c; stale longer extents are discarded and the new full extent is stored. Inputs are observed per-call API token totals, so text token estimation is not on the pricing critical path.

Calibration uses the 12 B6 native timelines and the 12 observed B6 mutation schedules. A second replay covers 54 real interactive sessions, up to 2,315 calls and 1.2 billion cumulative input tokens. Because sessions with native compaction violate the simple append-only model, results are reported for all 54 and for a frozen well-calibrated subset of 36 with absolute cache-creation error at most 20%. Policies are native, persistent-but-unaligned, cold-only, gated, and an oracle using the true remaining-call count.

We price calls in base-input-token equivalents (BITE):

$$\mathrm{BITE} = 0.1R + 2.0W + 1.0U + 5.0O. \tag{3}$$

Percentage deltas are more portable than absolute dollars because the interactive archive mixes models.

4.4 B8: preregistered live validation

B8 v2 contains three live pairs. Each session chains three graded Django tasks in one conversation, with a 60-second pause between tasks. The native arm is unchanged. The treatment combines admission with the persistent gated gateway. Before the first paid v2 run, the B7 machinery predicts a 29.5% pooled BITE reduction and freezes a validation band of 22–36% reduction. The primary endpoint is pooled BITE; secondary endpoints include CLI-reported cost, cumulative input, fires by reason, fallbacks, and nine task grades per arm.

An earlier v1 experiment is preserved as a confounded negative result. Its treatment used a custom base URL without admission and 65-minute gaps. The proxy caused the client to eagerly include tool schemas, and the nominal one-hour TTL remained warm. Per preregistration, post-hoc correction does not rescue v1; the findings instead determine the clean v2 design.

5 B6: Live Workload Reduction

Table 1 shows the primary result. Pooled cumulative input falls from 14,866,113 to 8,699,786 tokens, $R = 0.5852$, a 41.5% reduction. All four task-level pooled effects are positive, ranging from 28.1% to 58.7%. Ten of 12 individual pairs reduce input; stochastic trajectory length produces two reversals and a wide pair range. The post-hoc task-cluster bootstrap 95% interval is 31.1–54.8%. With only four clusters, this interval is a sensitivity summary rather than a basis for population generalization.

Objective quality is 10/12 native and 9/12 treatment, passing the frozen rule exactly at its bound. Task 16502 is the residual risk: treatment is 0/3 while native is 2/3. All four failures across arms share the same shallow-fix mode; one native and one treatment patch are essentially the same. The sample cannot rule out a treatment effect, and we do not use the cross-arm similarity to erase the failure. Task 16901 moves in the other direction, with treatment 3/3 and native 2/3.

Mechanism health is clean: 353 tool results are retired, 1,794 thinking blocks are stripped, and no mutated request falls back. Same-path repeat reads are 11/36 native and 12/39 treatment, providing no evidence of a retirement-induced reread tax. All runs receive real FAIL_TO_PASS and PASS_TO_PASS grading rather than process-success labels.

RQ1 (live): The integrated stack reduces cumulative input by 41.5% and passes the preregistered operational quality gate. The result is a context-workload claim in four Django tasks, not a universal token or quality estimate.

5.1 The dollar contradiction

B6 also exposes why cumulative input and dollars must be separate. Treatment makes 311 API calls versus 279 (+11.5%), lowers cache reads from 14,478,012 to 8,047,610 (44.4%), increases cache creation from 387,545 to 651,554 (68.1%), and increases output from 85,274 to 93,677 (9.9%). CLI cost moves only from $7.98 to $7.78, a 2.5% reduction. Repricing at the observed one-hour tier reconstructs a 2.8% reduction, nearly matching the CLI.

The cause is observable in the gateway schedule. Retirement fires only on batch boundaries; the full original history reappears between them. Each boundary pays another suffix creation and the retirement saving lasts one call. Thinking removal and admission remain applied on every call and carry most of the residency effect. This “flash” mutation is the failure that B7 persistence and

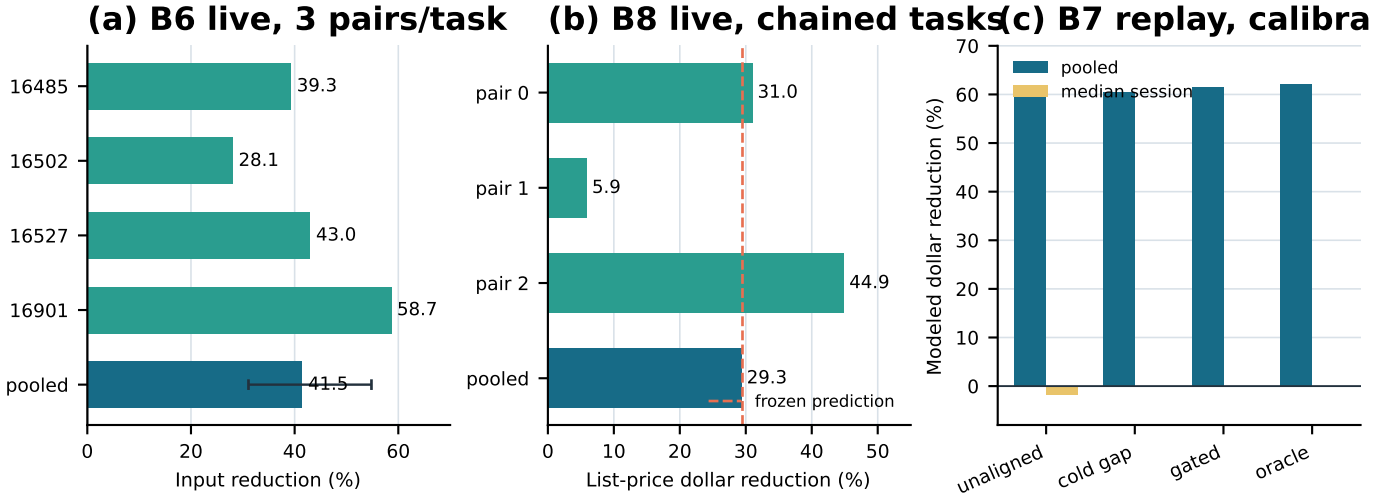

Figure 2: Three evidence stages. (a) B6 live cumulative-input reduction by task and pooled; the pooled whisker is a post-hoc four-task cluster-bootstrap 95% interval. (b) B8 v2 live BITE reduction by pair and pooled; dashed line is the frozen prediction. (c) B7 modeled dollar reduction on 36 well-calibrated interactive sessions. The pooled/median split shows that the modeled lifetime value is tail-concentrated. B7 values are not live savings.

Table 1: B6 preregistered primary endpoint and objective success counts. Input is the sum of cache read, cache creation, and uncached input over three runs per arm.

Task	Native input	Treatment input	Reduction	Success N/T
16485	2,772,225	1,682,341	39.3%	3/3
16502	4,753,322	3,418,567	28.1%	2/0
16527	3,606,230	2,055,051	43.0%	3/3
16901	3,734,336	1,543,827	58.7%	2/3
Pooled	14,866,113	8,699,786	41.5%	10/9

cache alignment repair.

6 B7: Cache Model and Scheduling

6.1 Calibration

Three platform facts are learned rather than assumed. First, the client requests the one-hour cache tier, so the write multiplier is 2.0 rather than 1.25. Second, the provider serves partial interior hits: editing at depth x preserves the matching prefix before x. Third, back-to-back sessions can begin with a warm shared fixed prefix. Incorporating these facts makes the append-only simulator exact on 11/12 B6 native sessions; the remaining session contains one retry anomaly among 279 native calls. Replaying the 12 observed treatment mutation schedules yields about 1% cache-read error and 7.3% median absolute cache-creation error. On the 54 interactive sessions, median absolute creation error is 3.0% but p90 is 64%, revealing a compaction-heavy regime outside the model.

6.2 Two workload regimes

On the 11 well-calibrated B6 native timelines, persistent unaligned mutation is modeled to increase dollars 8.4% while reducing residency 9.6%. Cold-only, gated, and oracle policies fire zero times and change cost by zero. Even an oracle finds no profitable mid-session edit in these short, hot sessions. Holding is the correct action.

On the 36 well-calibrated interactive sessions, the picture changes:

Table 2: B7 cache-policy replay on the well-calibrated interactive subset. All values are modeled.

Policy	Pooled Δ$	Median Δ$	Δres.	Fires
Unaligned	−61.9%	+1.9%	−74.2%	807
Cold gap	−60.4%	0.0%	−65.4%	125
Gated	−61.5%	0.0%	−67.0%	2,514
Oracle	−62.1%	0.0%	−73.7%	4,287

The pooled result is dominated by giant sessions. For gated scheduling, only 8/36 sessions have a positive modeled saving and 28 are unchanged; the top five sessions contribute about 90.4% of total modeled savings. The median session gains nothing. Unaligned scheduling reduces pooled cost yet regresses the median, exactly the behavior seen on B6-style timelines. Gated scheduling comes within 0.6 percentage points of the oracle's pooled cost while avoiding negative modeled deltas.

RQ2 (modeled): One policy adapts across regimes: hold in short cache-hot sessions, fire in the long tail when the read annuity dominates the rewrite. The 61.5% pooled interactive payoff remains a replay result, not a live claim.

Table 3: B8 v2 primary and secondary live effects. Each pair contains three chained tasks per arm; all 18 task instances pass objective grading.

Pair	Native BITE	Treatment BITE	Dollar reduction	Residency reduction
0	894,926	617,611	31.0%	43.5%
1	551,222	518,781	5.9%	11.0%
2	737,518	406,481	44.9%	55.9%
Pooled	2,183,665	1,542,873	29.3%	40.1%

7 B8: Preregistered Live Validation

Pooled BITE falls from 2,183,665 to 1,542,873, a 29.34% reduction. The frozen prediction is 29.5%, so error is −0.16 percentage points and the result is inside the preregistered 22–36% validation band. The independent CLI report falls from $6.553 to $4.638, or 29.22%. Cumulative input falls 40.10%, from 14,158,182 to 8,480,551. Pair-level dollar reductions are 31.0%, 5.9%, and 44.9%; reporting all three makes the replication variance visible. Quality is 9/9 in both arms.

No treatment request falls back. Each gateway fires once at cold start. The predicted marginal break-even fires do not occur, and no tool result or thinking block is retired mid-session. This is not a failure hidden by the aggregate: it is the scheduler's desired no-harm behavior under one-hour write pricing. The live cost saving is essentially all schema admission. The model predicts the aggregate accurately because lifetime's expected contribution in this shape was already small, but its individual fire-count forecast is wrong.

RQ3 (live): The admission-plus-gated stack cuts list-price cost 29.34%, within 0.16 points of the frozen prediction, and CLI cost independently agrees. The test validates admission pricing and cache-aware holding; it does not validate the B7 giant-session firing payoff.

7.1 The confounded v1 is part of the result

B8 v1 used a proxy without admission and 65-minute gaps. Its treatment moved outside the frozen band because setting a custom base URL disabled normal MCP tool-schema deferral. The first treatment request carried about 84,676 tokens versus 41,554 for native, followed by full-prefix rewrites as the tool list changed. One rewrite occurred when the gateway made no mutation, isolating the client behavior from the scheduler. Provider documentation now describes this fallback explicitly [2].

Native requests also retained cache hits after 65 minutes. Nominal TTL therefore did not identify a free boundary. Per the protocol, a post-hoc subtraction that would place one pair near the predicted band is not counted as validation. Instead, v1 yields two product rules: a proxy must preserve tool deferral or perform admission itself, and a time threshold alone is not proof that a cache is cold.

8 Discussion

8.1 The result is three claims, not one

The experiments support three different statements.

1. **Live workload:** admission plus unaligned lifetime control reduces cumulative input 41.5% in B6, with a frozen operational quality rule passed.
2. **Live dollars:** admission plus gated holding reduces BITE 29.34% and CLI cost 29.22% in B8; the realized saving is admission.
3. **Modeled lifetime dollars:** gated retirement reduces pooled cost about 61.5% in a calibrated replay of giant sessions, but the median is zero and no live firing-regime A/B exists.

Combining the largest percentages would be invalid. They refer to different workloads, policies, and evidence grades. The honest current product story is simpler: audit and shrink the fixed prefix first; observe cache behavior; enable persistent lifetime mutation only when Equation 1 clears; retain the original request as a safety path.

8.2 Why inaction is an optimization

Efficiency systems are often evaluated by how often they transform input. ContextRuntime's key behavior is sometimes to do nothing. On a hot prefix, each deleted token saves r on a future read but can force $w - r$ of new creation in the divergent suffix. With $w - r = 1.9$ and a conservative remaining-call estimate, most ordinary sessions cannot amortize the edit. A fixed threshold would mutate precisely when the cache makes mutation least attractive. Gated holding is therefore a positive result with a measurable counterfactual: it removes B6's modeled +8.4% cost regression and adds zero overhead to B8 beyond admission.

8.3 Provider portability

The runtime parameterizes TTL, read, write, and output multipliers. Under free or cheap cache writes, Equation 1 fires much more often; under expensive long-TTL writes it holds. The same session artifacts can be repriced, but provider profiles other than Anthropic one-hour are currently offline presets, not live validations. Cache-key construction, minimum cacheable length, eviction, and partial-hit semantics may differ, so portability requires a small calibration suite before enforcement.

8.4 Combining with learned compression

The planner currently replaces exact old payloads with recovery stubs. A learned compressor could reduce G earlier or preserve richer semantic state. The scheduler remains useful: any generated summary changes bytes and should be introduced when its expected read annuity clears the rewrite cost. Conversely, cache-aware prompt compression such as CAPC [24] could improve admission while the

retirement planner supplies object lifetime. The architectural boundary supports these compositions without treating a learned model as a safety proof.

9 Limitations and Future Work

Scale and scope. B6 has four task clusters and B8 three session pairs, all Django and one client/model family. Large within-task variance is visible. The operational quality gate is not a powered statistical non-inferiority test. Replication should expand repositories, languages, models, and agents while preserving paired prompts and objective grading.

The tail is not live. B7's roughly 60% pooled payoff appears in a few giant interactive sessions and is zero for the median. A decisive next experiment must enter that firing regime under controlled cost, preregister both pooled and distributional endpoints, and inspect recovery behavior.

Cache observability. The scheduler infers state from timing and a calibrated extent model. B8 v1 shows that nominal TTL is soft. Provider-supplied cache identifiers or explicit hit telemetry would make cold-boundary decisions more reliable. Until then, TTL-gap firing should use conservative margins or be disabled.

Token estimation off the critical path. Observed API usage prices sessions exactly, but pending retirement size and suffix depth use calibrated character/token estimators. Estimation error can change a knife-edge fire, as B8's predicted nine marginal fires versus zero observed illustrates. A safety margin or online calibration should accompany provider transfer.

Mutation semantics. Supersession is mechanically strong; cold-tail retirement is a policy assumption with a recovery path. Zero fallbacks across 17 live enforce sessions and no observed B6 reread tax are encouraging but do not prove semantic safety. Future work should log recovery-handle use, causal rereads, and failures over larger diverse workloads.

Private interactive traces. The 54-session B7 archive is not redistributed. Aggregate replay output and source code are public, but independent replication requires new traces. The live B6/B8 aggregate artifacts and gateway decision logs are committed.

10 Conclusion

Context optimization in coding agents is not a single compression problem. Fixed schemas, old tool results, hidden reasoning, future turns, and provider cache state interact. ContextRuntime separates admission, safety, mutation, and timing so that each can be measured independently. Live B6 results show a 41.5% cumulative-input reduction, but also reveal that unaligned edits turn cache reads into writes and nearly erase dollar savings. The calibrated B7 model turns this contradiction into a break-even scheduler. Live B8 then validates a 29.34% cost reduction against a frozen 29.5% prediction while the scheduler correctly refuses unprofitable mid-session work. Today, admission is the demonstrated dollar lever; cache-aligned lifetime savings in giant sessions remain a precise, testable hypothesis rather than a marketed result.

Artifact and Reproducibility Note

Code and artifacts: https://github.com/ankit961/LLM-Token-efficiency

The repository contains the B6 and B8 protocols, task-level live artifacts, treatment decision logs, B7 cache simulator, provider profiles, scheduler and gateway code, regression tests, and a script that recomputes all tables and figures without model calls. From the repository root, run `python3 papers/scripts/generate_assets.py`, then build this source with `latexmk -pdf main.tex`.

References

[1] Anthropic. Introducing advanced tool use on the claude developer platform. Engineering blog, 2025.

[2] Anthropic. Connect claude code to tools via MCP. Claude Code documentation, 2026.

[3] Anthropic. Manage tool context. Claude Platform documentation, 2026.

[4] Anthropic. Prompt caching. Claude Platform documentation, 2026.

[5] Ankit Chauhan. LLM Token Efficiency (ContextRuntime). GitHub repository, 2026. Evidence snapshot at commit 4656e8c.

[6] Yanfei Chen, Jinsung Yoon, Devendra Singh Sachan, Qingze Wang, Vincent Cohen-Addad, Mohammadhossein Bateni, Chen-Yu Lee, and Tomas Pfister. Re-invoke: Tool invocation rewriting for zero-shot tool retrieval. In *Findings of EMNLP*, pages 4705–4726, 2024.

[7] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of ACL*, pages 8697–8727, 2025.

[8] Xiang Fei, Xiawu Zheng, and Hao Feng. MCP-Zero: Proactive toolchain construction for LLM agents from scratch, 2025.

[9] Tao Ge, Hu Jing, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations*, 2024.

[10] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems*, volume 6, 2024.

[11] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing

prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, pages 13358–13376, 2023.

[12] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of ACL*, pages 1658–1677, 2024.

[13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[14] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. RAGCache: Efficient knowledge caching for retrieval-augmented generation, 2024.

[15] Maxim Khailo. Keeping the cache warm pays: Keepalive economics for agentic workloads, 2026.

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM Symposium on Operating Systems Principles*, 2023.

[17] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 929–945, 2024.

[18] Shukai Liu, Bo Jiang, Jian Yang, Yizhi LI, Jinyang Guo, Xianglong Liu, and Bryan Dai. Context as a tool: Context management for long-horizon SWE-agents. In *Findings of ACL*, pages 20604–20617, 2026.

[19] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. GraphCoder: Enhancing repository-level code completion via coarse-to-fine retrieval based on code context graph. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*, pages 570–581, 2024.

[20] Elias Lumer, Faheem Nizar, Akshaya Jangiti, Kevin Frank, Anmol Gulati, Mandar Phadate, and Vamse Kumar Subbiah. Don't break the cache: An evaluation of prompt caching for long-horizon agentic tasks, 2026.

[21] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*, 2023.

[22] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

[23] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of ACL*, pages 963–981, 2024.

[24] Yan Song. Cache-aware prompt compression: A two-tier cost model for LLM API caching, 2026.

[25] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. Preble: Efficient distributed prompt scheduling for LLM serving, 2024.

[26] Nikhil Verma. Active context compression: Autonomous memory management in LLM agents, 2026.

[27] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025.

[28] Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation. In *International Conference on Learning Representations*, 2024.

[29] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

[30] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of EuroSys*, 2025.

[31] Lu Ye, Ze Tao, Yong Huang, and Yang Li. ChunkAttention: Efficient self-attention with prefix-aware KV cache and two-phase partition. In *Proceedings of ACL*, 2024.

[32] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of EMNLP*, pages 2471–2484, 2023.

[33] Xuan Zhang, Longtao Zheng, Cunxiao Du, Bo An, and Xin Dong. AutoCompact: Learning when to compact context in long-horizon coding agents. Project report, 2026.

[34] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs, 2024.

[35] Yuanhang Zheng, Peng Li, Wei Liu, Yang Liu, Jian Luan, and Bin Wang. ToolRerank: Adaptive and hierarchy-aware reranking for tool retrieval. In *Proceedings of LREC-COLING*, pages 16263–16273, 2024.